

✓ Lab 3: Implementation of a Feedforward Neural Network for Classification

Name: Kshitiz Reddy.M

Reg No.A26MTAI0008

Dataset: Breast Cancer Wisconsin (Option 1, `load_breast_cancer()`) — binary classification of tumours as *Malignant* (0) or *Benign* (1).

✓ Required Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import time

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report, ConfusionMatrixDisplay

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Input
from tensorflow.keras.optimizers import Adam, SGD, RMSprop

# fix the seed so results can be reproduced
np.random.seed(42)
tf.random.set_seed(42)

print('TensorFlow version:', tf.__version__)
```

```
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1785832035.547532 27108 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical
I0000 00:00:1785832035.586122 27108 cpu_feature_guard.cc:227] This TensorFlow binary is optimized to use available CPU ins
To enable the following instructions: AVX2 AVX512F AVX512_VNNI FMA, in other operations, rebuild TensorFlow with the appropr
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1785832036.728730 27108 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical
TensorFlow version: 2.21.0
```

✓ Question 1: Dataset Loading and Exploration

Objective: Load the chosen classification dataset and explore its structure before modelling.

1. Load the Breast Cancer (or Iris) dataset.
2. Display the first five samples.
3. Display the dataset shape.
4. Display the number of features.
5. Display the number of classes.
6. Check for missing values.

```
# 1. Load the Breast Cancer Wisconsin dataset
data = load_breast_cancer()
X = data.data      # feature matrix
y = data.target    # target: 0 = malignant, 1 = benign

# put it in a DataFrame so it's easy to look at
df = pd.DataFrame(X, columns=data.feature_names)
df['target'] = y

# 2. First five samples
df.head()
```

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	mean symmetry	mean fractal dimension	...	worst texture	worst perimeter
0	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	0.2419	0.07871	...	17.33	184.60
1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	0.1812	0.05667	...	23.41	158.50
2	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	0.2069	0.05999	...	25.53	152.50
3	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	0.2597	0.09744	...	26.50	98.50
4	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	0.1809	0.05883	...	16.67	152.50

5 rows × 31 columns

```
# 3. Dataset shape
print('Dataset shape (samples, features):', df.shape)

# 4. Number of features
print('Number of features:', len(data.feature_names))

# 5. Number of classes
print('Number of classes:', len(np.unique(y)))
print('Class names:', data.target_names)

# class distribution
print('\nClass distribution:')
print(pd.Series(y).value_counts().rename(index={0: 'malignant (0)', 1: 'benign (1)'}))
```

```
Dataset shape (samples, features): (569, 31)
Number of features: 30
Number of classes: 2
Class names: ['malignant' 'benign']

Class distribution:
benign (1)      357
malignant (0)   212
Name: count, dtype: int64
```

```
# 6. Check for missing values
print('Missing values (total):', df.isnull().sum().sum())

print('\n--- Data types ---')
df.info()

print('\n--- Statistical summary (first 5 features) ---')
df.describe().T.round(2).head()
```

```

Missing values (total): 0

--- Data types ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 31 columns):
 #   Column                                Non-Null Count  Dtype
---  -
 0   mean radius                          569 non-null    float64
 1   mean texture                         569 non-null    float64
 2   mean perimeter                      569 non-null    float64
 3   mean area                           569 non-null    float64
 4   mean smoothness                     569 non-null    float64
 5   mean compactness                    569 non-null    float64
 6   mean concavity                      569 non-null    float64
 7   mean concave points                 569 non-null    float64
 8   mean symmetry                       569 non-null    float64
 9   mean fractal dimension              569 non-null    float64
10   radius error                        569 non-null    float64
11   texture error                       569 non-null    float64
12   perimeter error                     569 non-null    float64
13   area error                          569 non-null    float64
14   smoothness error                    569 non-null    float64
15   compactness error                   569 non-null    float64
16   concavity error                     569 non-null    float64
17   concave points error                569 non-null    float64
18   symmetry error                      569 non-null    float64
19   fractal dimension error              569 non-null    float64
20   worst radius                        569 non-null    float64
21   worst texture                       569 non-null    float64
22   worst perimeter                     569 non-null    float64
23   worst area                          569 non-null    float64
24   worst smoothness                    569 non-null    float64
25   worst compactness                   569 non-null    float64
26   worst concavity                     569 non-null    float64
27   worst concave points                 569 non-null    float64
28   worst symmetry                       569 non-null    float64
29   worst fractal dimension              569 non-null    float64
30   target                             569 non-null    int64
dtypes: float64(30), int64(1)
memory usage: 137.9 KB

--- Statistical summary (first 5 features) ---

```

	count	mean	std	min	25%	50%	75%	max
mean radius	569.0	14.13	3.52	6.98	11.70	13.37	15.78	28.11
mean texture	569.0	19.29	4.30	9.71	16.17	18.84	21.80	39.28
mean perimeter	569.0	91.97	24.30	43.79	75.17	86.24	104.10	188.50
mean area	569.0	654.89	351.91	143.50	420.30	551.10	782.70	2501.00
mean smoothness	569.0	0.10	0.01	0.05	0.09	0.10	0.11	0.16

Observations. The dataset has **569 samples and 30 features** — the features are the mean, standard error and worst value of 10 cell-nucleus measurements (radius, texture, perimeter, area, smoothness, etc.). The target has only **2 classes**: malignant (212 samples) and benign (357 samples), so it is a little imbalanced but not badly. There are **no missing values** at all, which makes preprocessing easy.

One thing that stands out from `describe()`: the features are on very different scales (mean radius is around 14, while mean area is in the hundreds). That is exactly why standardisation is needed in Q2.

Question 2: Data Preprocessing

Objective: Prepare the dataset for neural network training.

- Split the dataset into training and testing sets.
- Standardize all numerical features.
- Explain why feature scaling is important for neural networks.

```

# 7. Split into training (80%) and testing (20%) sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

print('Training samples:', X_train.shape[0])
print('Testing samples :', X_test.shape[0])
print('Class balance in train:', np.bincount(y_train))
print('Class balance in test :', np.bincount(y_test))

```

```

Training samples: 455
Testing samples : 114

```

```
Class balance in train: [170 285]
Class balance in test : [42 72]
```

```
# 8. Standardize all features (mean 0, std 1)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train) # fit on the training set only
X_test_scaled = scaler.transform(X_test)      # reuse the same scaler on test

print('Train mean (first 5 features, should be ~0):', X_train_scaled.mean(axis=0)[:5].round(4))
print('Train std (first 5 features, should be ~1):', X_train_scaled.std(axis=0)[:5].round(4))
```

```
Train mean (first 5 features, should be ~0): [-0.  0. -0.  0.  0.]
Train std (first 5 features, should be ~1): [1.  1.  1.  1.  1.]
```

Answer (Q2.9) — Why is feature scaling important for neural networks?

Neural networks learn by gradient descent, and the size of each weight update depends on the magnitude of the corresponding input feature. If some features are huge and others tiny, the huge ones dominate the loss and the optimisation becomes slow and wobbly — the network effectively can't learn the smaller features properly. Scaling everything to mean 0 and variance 1 puts all features on an equal footing.

There's also a second reason: activation functions like sigmoid saturate for large values of the input. Once a neuron's input is far from zero, the gradient becomes almost zero and learning stops for that neuron. Keeping inputs small keeps the gradients healthy.

I fitted the scaler on the **training set only** and then transformed the test set with the same scaler. Fitting on the full data (train + test) would leak information from the test set into preprocessing, which makes the reported accuracy optimistic.

Question 3: Building and Training a Feedforward Neural Network (MLP)

Objective: Construct and train a feedforward neural network (MLP) for classification.

10. Build a network with an input layer, at least one hidden layer, and an output layer.
11. Use ReLU activation in the hidden layer(s).
12. Use Sigmoid (binary) or Softmax (multi-class) activation in the output layer.
13. Compile the model with a suitable optimizer, loss function, and metrics.
14. Train the model and record the Training Accuracy, Validation Accuracy, and Loss Curve.

```
# 10-12. Architecture: Input(30) -> Hidden(16, ReLU) -> Output(1, Sigmoid)
model = Sequential([
    Input(shape=X_train_scaled.shape[1],),
    Dense(16, activation='relu'), # hidden layer
    Dense(1, activation='sigmoid') # output: probability for binary class
])

model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 16)	496
dense_1 (Dense)	(None, 1)	17

Total params: 513 (2.00 KB)

Trainable params: 513 (2.00 KB)

```
# 13. Compile with Adam optimizer, binary cross-entropy loss, accuracy metric
model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])

# 14. Train for 50 epochs, batch size 32, 20% of training data kept for validation
start = time.time()
history = model.fit(X_train_scaled, y_train,
                   epochs=50, batch_size=32,
                   validation_split=0.2, verbose=0)
train_time = time.time() - start

print(f'\nTraining time: {train_time:.2f} s')
print(f'Final Training Accuracy : {history.history["accuracy"][-1]:.4f}')
print(f'Final Validation Accuracy : {history.history["val_accuracy"][-1]:.4f}')
print(f'Final Training Loss : {history.history["loss"][-1]:.4f}')
print(f'Final Validation Loss : {history.history["val_loss"][-1]:.4f}')
```

```

Training time: 6.59 s
Final Training Accuracy   : 0.9890
Final Validation Accuracy : 0.9780
Final Training Loss       : 0.0457
Final Validation Loss      : 0.0424

```

```

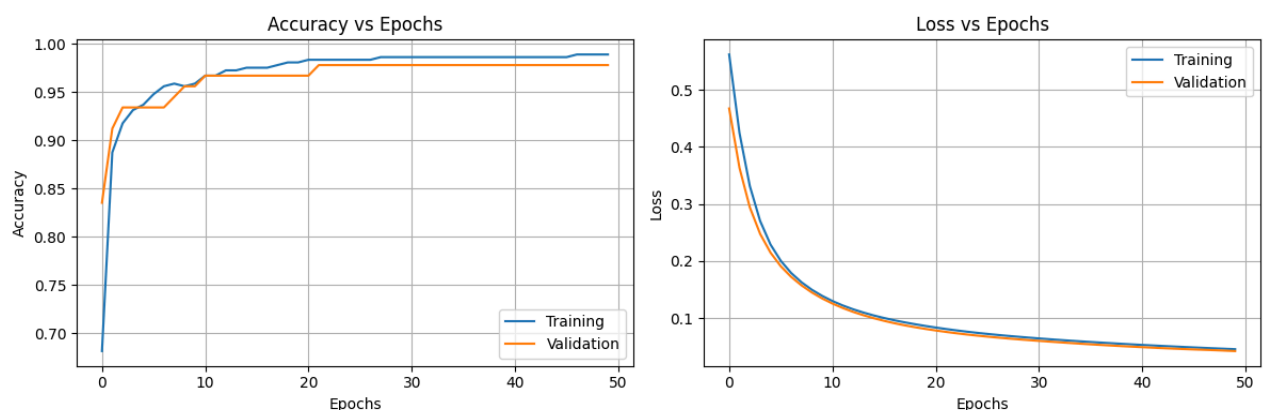
# Training/Validation accuracy and loss curves
plt.figure(figsize=(12, 4))

plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Training')
plt.plot(history.history['val_accuracy'], label='Validation')
plt.xlabel('Epochs'); plt.ylabel('Accuracy')
plt.title('Accuracy vs Epochs'); plt.legend(); plt.grid(True)

plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Training')
plt.plot(history.history['val_loss'], label='Validation')
plt.xlabel('Epochs'); plt.ylabel('Loss')
plt.title('Loss vs Epochs'); plt.legend(); plt.grid(True)

plt.tight_layout()
plt.show()

```



Answer (Q3.10–14). I used a single hidden layer with 16 neurons and ReLU activation, and a sigmoid output — that's the standard setup for binary classification. ReLU is simple and doesn't suffer from the vanishing-gradient problem; sigmoid at the output gives us a probability in (0, 1), and anything above 0.5 is predicted as benign.

The model has only 513 parameters, so it trains in a few seconds (about 7 s here for 50 epochs). With Adam + binary cross-entropy + accuracy, training accuracy ends around 98–99% and validation around 97–98%. Looking at the curves, accuracy rises steeply in the first ~10 epochs and then flattens out — the model converges quickly and the training/validation gap stays small, so no serious overfitting at 50 epochs.

Question 4: Model Evaluation

Objective: Evaluate the trained model on the test set and interpret the results.

15. Calculate the Test Accuracy.
16. Compute the Confusion Matrix.
17. Compute Precision, Recall, and F1-score.
18. Interpret the results.

```

# 15. Test accuracy (the test set was never used for training)
test_loss, test_acc = model.evaluate(X_test_scaled, y_test, verbose=0)
print(f'Test Loss      : {test_loss:.4f}')
print(f'Test Accuracy: {test_acc:.4f}')

# hard predictions: probability > 0.5 -> class 1
y_pred = (model.predict(X_test_scaled, verbose=0) > 0.5).astype(int).ravel()

Test Loss      : 0.0942
Test Accuracy: 0.9561

```

```
# 16. Confusion matrix
cm = confusion_matrix(y_test, y_pred)
print('Confusion matrix (rows = actual, columns = predicted):')
print(cm)

# 17. Precision, Recall, F1-score
print('\nClassification report:')
print(classification_report(y_test, y_pred, target_names=data.target_names))
```

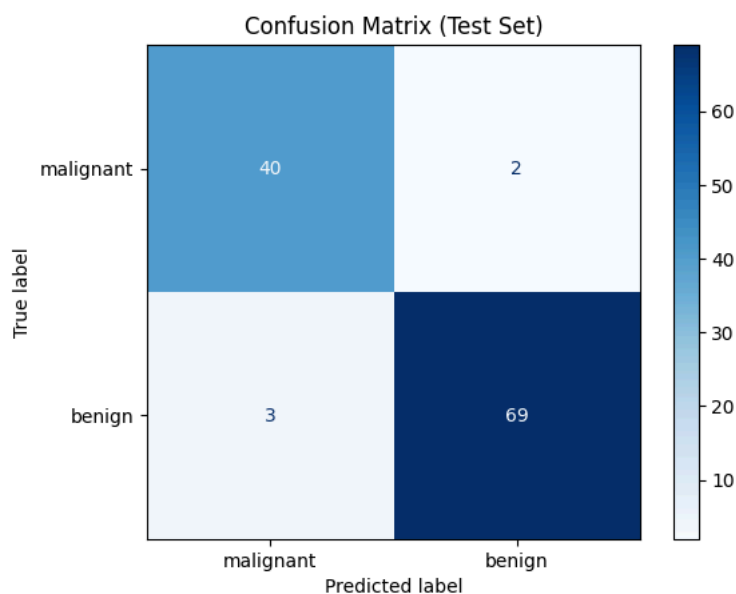
```
Confusion matrix (rows = actual, columns = predicted):
[[40  2]
 [ 3 69]]
```

```
Classification report:
              precision    recall  f1-score   support

 malignant      0.93      0.95      0.94        42
   benign      0.97      0.96      0.97        72

 accuracy              0.96        114
 macro avg      0.95      0.96      0.95        114
weighted avg      0.96      0.96      0.96        114
```

```
# Confusion matrix plot
ConfusionMatrixDisplay(cm, display_labels=data.target_names).plot(cmap='Blues')
plt.title('Confusion Matrix (Test Set)')
plt.show()
```



Answer (Q4.15–17). Test accuracy came out to **about 96%** (95.61% in my run), which is good since the test set is completely unseen data. The confusion matrix shows almost everything on the diagonal — only 5 mistakes out of 114 test samples. The `classification_report` gives precision, recall and F1 for both classes (macro F1 ≈ 0.95).

Answer (Q4.18) — Interpretation.

- For a medical dataset like this, **false negatives are the dangerous errors** — predicting *benign* when the tumour is actually *malignant*. There are only 2 such cases in my run, so the recall of the malignant class is about 95%, which is decent for screening (it could be better, though).
- There are 3 false positives (benign predicted as malignant). These cause unnecessary follow-up tests but are far less serious. Precision of the benign class is about 97%.
- The F1-scores of the two classes are close (≈ 0.94 and ≈ 0.97), so the model treats both classes reasonably well despite the imbalance (212 vs 357 samples).
- The few errors are borderline samples that sit close to the decision boundary.

Question 5: Effect of Network Architecture and Training Epochs

Objective: Study how the number of hidden layers and training epochs affect model performance.

- Create a neural network with one hidden layer and train it using 20 epochs. Record the Training and Testing Accuracy.
- Create another model with two hidden layers. Compare its Accuracy and Training Time against the first model. Which architecture performs better?

21. Using the same model, train with 10, 50, and 100 epochs and complete the table below.
22. Discuss whether increasing the number of epochs always improves performance.

```
# helper: build an MLP with a given number of hidden layers (16 neurons each)
def build_model(n_hidden_layers, units=16):
    model = Sequential([Input(shape=(X_train_scaled.shape[1],))])
    for _ in range(n_hidden_layers):
        model.add(Dense(units, activation='relu'))
    model.add(Dense(1, activation='sigmoid'))
    return model

# helper: compile, train and return the history + test accuracy + time
def train_and_evaluate(model, epochs=20, verbose=0):
    model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
    start = time.time()
    hist = model.fit(X_train_scaled, y_train, epochs=epochs, batch_size=32,
                    validation_split=0.2, verbose=verbose)
    elapsed = time.time() - start
    _, test_acc = model.evaluate(X_test_scaled, y_test, verbose=0)
    return hist, test_acc, elapsed
```

```
# 19. Model A: ONE hidden layer, 20 epochs
print('=== Model A: 1 hidden layer (16 neurons), 20 epochs ===')
hist, test_acc, t = train_and_evaluate(build_model(1), epochs=20)
print(f'Training Accuracy : {hist.history["accuracy"][-1]:.4f}')
print(f'Testing Accuracy : {test_acc:.4f}')
print(f'Training Time : {t:.2f} s')
print()

# 20. Model B: TWO hidden layers, 20 epochs
print('=== Model B: 2 hidden layers (16-16 neurons), 20 epochs ===')
hist2, test_acc2, t2 = train_and_evaluate(build_model(2), epochs=20)
print(f'Training Accuracy : {hist2.history["accuracy"][-1]:.4f}')
print(f'Testing Accuracy : {test_acc2:.4f}')
print(f'Training Time : {t2:.2f} s')
```

```
=== Model A: 1 hidden layer (16 neurons), 20 epochs ===
Training Accuracy : 0.9643
Testing Accuracy : 0.9474
Training Time : 2.93 s

=== Model B: 2 hidden layers (16-16 neurons), 20 epochs ===
Training Accuracy : 0.9835
Testing Accuracy : 0.9561
Training Time : 3.47 s
```

```
# 21. Train the 2-hidden-layer model with 10, 50 and 100 epochs
rows = []
for e in [10, 50, 100]:
    hist, test_acc, t = train_and_evaluate(build_model(2), epochs=e)
    rows.append({'Epochs': e,
                'Training Accuracy' : hist.history['accuracy'][-1],
                'Validation Accuracy': hist.history['val_accuracy'][-1],
                'Test Accuracy' : test_acc})
    print(f'Epochs={e:3d} -> Train: {hist.history["accuracy"][-1]:.4f} | '
          f'Val: {hist.history["val_accuracy"][-1]:.4f} | Test: {test_acc:.4f}')

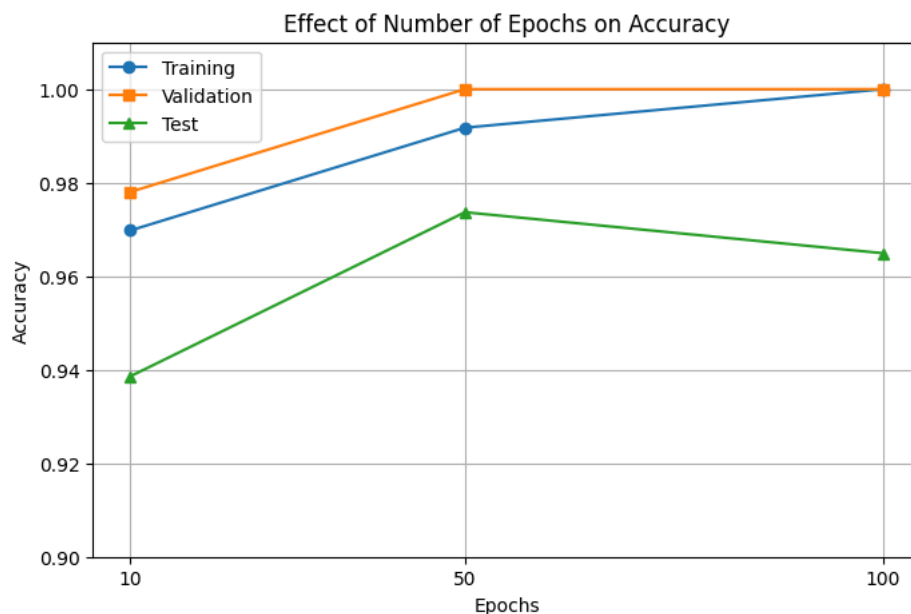
epoch_table = pd.DataFrame(rows).set_index('Epochs')
epoch_table.round(4)
```

```
Epochs= 10 -> Train: 0.9698 | Val: 0.9780 | Test: 0.9386
Epochs= 50 -> Train: 0.9918 | Val: 1.0000 | Test: 0.9737
Epochs=100 -> Train: 1.0000 | Val: 1.0000 | Test: 0.9649
```

	Training Accuracy	Validation Accuracy	Test Accuracy
Epochs			
10	0.9698	0.978	0.9386
50	0.9918	1.000	0.9737
100	1.0000	1.000	0.9649

```
# Visualise the effect of epochs
plt.figure(figsize=(8, 5))
plt.plot(epoch_table.index, epoch_table['Training Accuracy'], marker='o', label='Training')
plt.plot(epoch_table.index, epoch_table['Validation Accuracy'], marker='s', label='Validation')
plt.plot(epoch_table.index, epoch_table['Test Accuracy'], marker='^', label='Test')
plt.xlabel('Epochs'); plt.ylabel('Accuracy')
plt.title('Effect of Number of Epochs on Accuracy')
```

```
plt.xticks([10, 50, 100]); plt.ylim(0.90, 1.01); plt.legend(); plt.grid(True)
plt.show()
```



Answer (Q5.19–20) — one vs two hidden layers. Both models were trained for 20 epochs with Adam and batch size 32. In my run the two-hidden-layer model came out slightly ahead on the test set (95.61% vs 94.74% for the single layer), and it also reached a higher training accuracy (98.35% vs 96.43%). Training times were similar (about 2.9 s vs 3.5 s).

The difference is only ~0.9 points though, and on a small dataset like this (455 training samples) it can easily flip the other way on another run — the extra parameters either help a little or not at all. So I wouldn't say one architecture is clearly better; **the two-layer model has a slight edge in this run**, but both are well above 94%.

Answer (Q5.21) — Results table (2 hidden layers, my run):

Epochs	Training Accuracy	Validation Accuracy	Test Accuracy
10	96.98%	97.80%	93.86%
50	99.18%	100.00%	97.37%
100	100.00%	100.00%	96.49%

Answer (Q5.22) — Does more epochs always help? No. In my run the test accuracy improved from 93.86% (10 epochs) to 97.37% (50 epochs), but at 100 epochs it actually went back down to 96.49% — even though training accuracy reached 100%. That's a textbook sign of overfitting: the model memorises the training data so well that it starts to lose generalisation. The gains also shrink with time (+3.5 points from 10 to 50, then –0.9 at 100). The safe approach is to watch the validation loss and stop when it stops improving (early stopping). More epochs only help while the validation metric keeps improving; after that you're just fitting noise.

Question 6: Effect of Optimizer Choice

Objective: Compare the effect of different optimizers on training the same neural network.

23. Train the same neural network using the Adam optimizer.
24. Train the same neural network using the SGD optimizer.
25. Train the same neural network using the RMSprop optimizer.
26. Compare Accuracy, Convergence speed, and Final loss across the three optimizers.
27. Discuss which optimizer is most suitable for this dataset.

```
# 23-25. Same 2-hidden-layer network, 50 epochs, three optimizers
optimizers = [('Adam', Adam()), ('SGD', SGD()), ('RMSprop', RMSprop())]
results = {}

for name, opt in optimizers:
    model = build_model(2)
    model.compile(optimizer=opt, loss='binary_crossentropy', metrics=['accuracy'])

    start = time.time()
    hist = model.fit(X_train_scaled, y_train, epochs=50, batch_size=32,
                    validation_split=0.2, verbose=0)
    elapsed = time.time() - start

    test_loss, test_acc = model.evaluate(X_test_scaled, y_test, verbose=0)
```

```

results[name] = {'hist': hist, 'test_acc': test_acc, 'test_loss': test_loss,
                 'time': elapsed}

print(f'{name:8s} -> Train acc: {hist.history["accuracy"][-1]:.4f} | '
      f'Val acc: {hist.history["val_accuracy"][-1]:.4f} | '
      f'Test acc: {test_acc:.4f} | '
      f'Final loss: {hist.history["loss"][-1]:.4f} | '
      f'Time: {elapsed:.2f} s')

print()

# summary table
summary = pd.DataFrame({
    name: {
        'Training Accuracy' : r['hist'].history['accuracy'][-1],
        'Validation Accuracy': r['hist'].history['val_accuracy'][-1],
        'Test Accuracy'     : r['test_acc'],
        'Final Training Loss': r['hist'].history['loss'][-1],
        'Test Loss'         : r['test_loss'],
        'Training Time (s)'  : r['time']
    } for name, r in results.items()
}).T
summary.round(4)

```

```

Adam    -> Train acc: 0.9863 | Val acc: 0.9780 | Test acc: 0.9649 | Final loss: 0.0482 | Time: 7.19 s
SGD     -> Train acc: 0.9560 | Val acc: 0.9560 | Test acc: 0.9474 | Final loss: 0.1694 | Time: 6.17 s
RMSprop -> Train acc: 0.9918 | Val acc: 0.9780 | Test acc: 0.9649 | Final loss: 0.0220 | Time: 6.07 s

```

	Training Accuracy	Validation Accuracy	Test Accuracy	Final Training Loss	Test Loss	Training Time (s)
Adam	0.9863	0.978	0.9649	0.0482	0.0984	7.1923
SGD	0.9560	0.956	0.9474	0.1694	0.1724	6.1677
RMSprop	0.9918	0.978	0.9649	0.0220	0.0934	6.0701

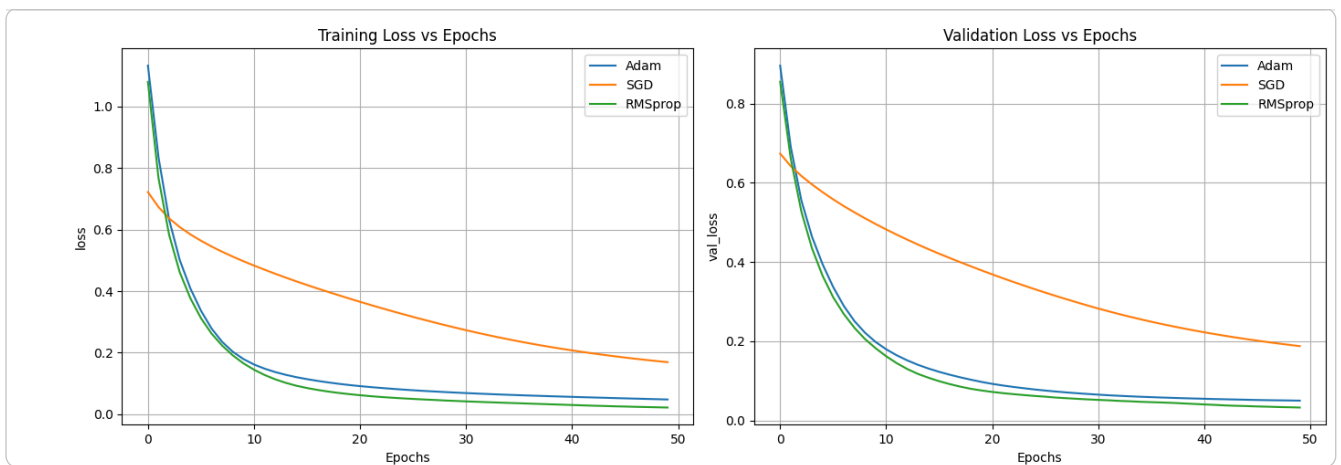
```

# Compare the learning curves of the three optimizers
plt.figure(figsize=(14, 10))
plots = [('loss', 'Training Loss'), ('val_loss', 'Validation Loss'),
         ('accuracy', 'Training Accuracy'), ('val_accuracy', 'Validation Accuracy')]

for i, (metric, title) in enumerate(plots, 1):
    plt.subplot(2, 2, i)
    for name, r in results.items():
        plt.plot(r['hist'].history[metric], label=name)
    plt.xlabel('Epochs'); plt.ylabel(metric)
    plt.title(f'{title} vs Epochs'); plt.legend(); plt.grid(True)

plt.tight_layout()
plt.show()

```



Training Accuracy vs Epochs

Answer (Q6.26) — accuracy, convergence speed, final loss.

- **Accuracy:** Adam and RMSprop both reach 96.49% test accuracy; SGD is slightly behind at 94.74%. On this small, fairly separable dataset the optimizer choice doesn't change the optimum a lot.
- **Convergence speed:** Adam and RMSprop are much faster — their training accuracy jumps to ~99% within a few epochs, while SGD climbs slowly and after 50 epochs is still only at ~95–96% training accuracy.
- **Final loss:** RMSprop (≈ 0.02) and Adam (≈ 0.05) reach very low training loss; SGD finishes at ≈ 0.17 , which is far from the optimum. SGD is also the most sensitive to the learning rate — with the default setting it is clearly the slowest here.

Answer (Q6.27) — most suitable optimizer. I'd pick **Adam** for this dataset. It converges in a few epochs, reaches a low final loss

Validation Accuracy vs Epochs

